- as: Specifies the content type you want returned, especially if that content type is :json.
- env: A hash of environment variables. I admit it's not immediately clear to me why you'd want to set those in a request spec.
- headers: A hash of simulated header information you might want passed.
- params: The params hash for controllers, including anything that might be in a query string or a route variable. You can use the params hash to cover items that might otherwise be arguments to the URL.
- xhr: If true, tells the controller to treat the fake request as an Ajax call.

So, a contrived call might look like this:

```
get(projects_url, params: {id: @project.id}, xhr: true, as: :json)
```

If one of the param arguments is an uploaded file—say, from a multipart form—you can simulate that using the Rails helper fixture_file_upload(filename, mime_type), like this:

```
post(:create,
  params: {logo: fixture_file_upload('/test/data/logo.png', 'image/png')}
```

Third-party tools to manage uploads, such as Paperclip and CarrierWave, typically have more specific testing helpers.

If you've done Rails controller tests before, you'll notice that this is an API change. Most notably, you can no longer set the session or the flash via request tests. Rails now considers those to be internal implementation details.

What to Expect in a Request Spec

My main goal in a request spec is to test behavior. I find that test-double validations work well here. If you're interested in testing the final state after the request, in addition to testing the changes to the database, there are a few other substantive things you can test.

After the request has been made, you have access to a response variable. Since request specs wrap Rails integration tests, RSpec does render the view, though if you are testing view behavior, I recommend using system specs, where you have access to the Capybara matchers, and not just the Rails integration spec matchers as described in Chapter 12, *Minitest*, on page 241.

After the request has been made, you have access to three special hash variables that you can test—cookies, flash, and session—all of which test the values the Rails process stored in those buckets.